Scrape data from HTML tables into a DataFrame using BeautifulSoup and Pandas

[57]:
```
import pandas as pd
```

[58]:
```
#The below url contains html tables with data about world population.
url = "https://en.wikipedia.org/wiki/World_population"
```

[33]:
```
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                  "AppleWebKit/537.36 (KHTML, like Gecko) "
                  "Chrome/91.0.4472.124 Safari/537.36"
}
data  = requests.get(url,headers=headers)
data.headers.get("content-type")
```

[33]:
```
'text/html; charset=UTF-8'
```

[35]:
```
soup = BeautifulSoup(data.text,"html.parser")
```

[36]:
```
#find all html tables in the web page
tables = soup.find_all('table') # in html table is represented by the tag <table>
```

[37]:
```
# we can see how many tables are found
len(tables)
```

[37]:
```
27
```

[38]:
```
for index,table in enumerate(tables):
    if ("10 most densely populated countries" in str(table)):
        table_index = index
print(table_index)
```

```
5
```

[44]:
```
print(tables[table_index].prettify())
```

[45]:
```
population_data = pd.DataFrame(columns=["Rank", "Country", "Population", "Area", "Density"])

for row in tables[table_index].tbody.find_all("tr"):
    col = row.find_all("td")
    if col:
        rank = col[0].text.strip()
        country = col[1].text.strip()
        population = col[2].text.strip()
        area = col[3].text.strip()
        density = col[4].text.strip()

        # Create a temporary DataFrame for the new row
        new_row = pd.DataFrame([{"Rank": rank, "Country": country, "Population": population, "Area": area, "Density": density}])

        # Use concat
        population_data = pd.concat([population_data, new_row], ignore_index=True)

population_data
```

[45]:

	Rank	Country	Population	Area	Density
0	1	Singapore	5,921,231	719	8,235
1	2	Bangladesh	165,650,475	148,460	1,116
2	3	Palestine[note 3][102]	5,223,000	6,025	867
3	4	Taiwan[note 4]	23,580,712	35,980	655
4	5	South Korea	51,844,834	99,720	520
5	6	Lebanon	5,296,814	10,400	509
6	7	Rwanda	13,173,730	26,338	500
7	8	Burundi	12,696,478	27,830	456
8	9	Israel	9,402,617	21,937	429
9	10	India	1,389,637,446	3,287,263	423

Scrape data from HTML tables into a DataFrame using BeautifulSoup and read_html

[59]:
```
from io import StringIO
pd.read_html(StringIO(str(tables[5])), flavor='bs4')
```

[59]:
```
[   Rank                Country  Population  Area (km2)  Density (pop/km2)
 0     1              Singapore     5921231         719               8235
 1     2             Bangladesh   165650475      148460               1116
 2     3  Palestine[note 3][102]     5223000        6025                867
 3     4          Taiwan[note 4]    23580712       35980                655
 4     5             South Korea    51844834       99720                520
 5     6                 Lebanon     5296814       10400                509
 6     7                  Rwanda    13173730       26338                500
 7     8                 Burundi    12696478       27830                456
 8     9                  Israel     9402617       21937                429
 9    10                   India  1389637446     3287263                423]
```

The function `read_html` always returns a list of DataFrames so we must pick the one we want out of the list.

[60]:
```
population_data_read_html = pd.read_html(StringIO(str(tables[5])), flavor='bs4')[0]

population_data_read_html
```

[60]:

	Rank	Country	Population	Area (km2)	Density (pop/km2)
0	1	Singapore	5921231	719	8235
1	2	Bangladesh	165650475	148460	1116
2	3	Palestine[note 3][102]	5223000	6025	867
3	4	Taiwan[note 4]	23580712	35980	655
4	5	South Korea	51844834	99720	520
5	6	Lebanon	5296814	10400	509
6	7	Rwanda	13173730	26338	500
7	8	Burundi	12696478	27830	456
8	9	Israel	9402617	21937	429
9	10	India	1389637446	3287263	423

Scrape data from HTML tables into a DataFrame using read_html

We can also use the `read_html` function to directly get DataFrames from a `url`.

[28]:
```
dataframe_list = pd.read_html(StringIO(data.text), flavor='bs4')
```

We can see there are 26 DataFrames just like when we used `find_all` on the `soup` object.

[29]:
```
len(dataframe_list)
```

[29]:
```
27
```

[45]:
```
print (dataframe_list[6])
print (dataframe_list[5])
```

```
   Rank         Country  Population  Area (km2)  Density (pop/km2)  \
0     1           India  1389637446     3287263                423
1     2        Pakistan   242923845      796095                305
2     3      Bangladesh   165650475      148460               1116
3     4           Japan   124214766      377915                329
4     5     Philippines   114597229      300000                382
5     6         Vietnam   103808319      331210                313
6     7  United Kingdom    67791400      243610                278
7     8     South Korea    51844834       99720                520
8     9          Taiwan    23580712       35980                655
9    10       Sri Lanka    23187516       65610                353

  Population trend[citation needed]
0                           Growing
1                     Rapidly growing
2                           Growing
3                    Declining[103]
4                           Growing
5                           Growing
6                           Growing
7                         Declining
8                         Declining
9                           Growing
   Rank                 Country  Population  Area (km2)  Density (pop/km2)
0     1               Singapore     5921231         719               8235
1     2              Bangladesh   165650475      148460               1116
2     3  Palestine[note 3][102]     5223000        6025                867
3     4          Taiwan[note 4]    23580712       35980                655
4     5             South Korea    51844834       99720                520
5     6                 Lebanon     5296814       10400                509
6     7                  Rwanda    13173730       26338                500
7     8                 Burundi    12696478       27830                456
8     9                  Israel     9402617       21937                429
9    10                   India  1389637446     3287263                423
```

[31]:
```
heading = soup.find("h3", {"id": "Most_densely_populated_countries"})

# Get the next table after this heading
table = heading.find_next("table")

# Convert to DataFrame
df = pd.read_html(StringIO(str(table)))[0]
print(df.head())
```

```
   Rank                 Country  Population  Area (km2)  Density (pop/km2)
0     1               Singapore     5921231         719               8235
1     2              Bangladesh   165650475      148460               1116
2     3  Palestine[note 3][102]     5223000        6025                867
3     4          Taiwan[note 4]    23580712       35980                655
4     5             South Korea    51844834       99720                520
```

[57]:
```
heading = soup.find("h3", {"id": "Most_densely_populated_countries"})

# Get the next table after this heading
table = heading.find_next("table")

# Convert to DataFrame
df = pd.read_html(StringIO(str(table)))[0]
df.head()
```

[57]:

	Rank	Country	Population	Area (km2)	Density (pop/km2)
0	1	Singapore	5921231	719	8235
1	2	Bangladesh	165650475	148460	1116
2	3	Palestine[note 3][102]	5223000	6025	867
3	4	Taiwan[note 4]	23580712	35980	655
4	5	South Korea	51844834	99720	520

Downloading And Scraping The Contents Of A Web Page

We Download the contents of the web page:

[47]:
```
url = "https://web.archive.org/web/20230224123642/https://www.ibm.com/us-en/"
```

We use `get` to download the contents of the webpage in text format and store in a variable called `data`:

[48]:
```
data  = requests.get(url).text
```

We create a `BeautifulSoup` object using the `BeautifulSoup` constructor

[49]:
```
soup = BeautifulSoup(data,"html.parser")  # create a soup object using the variable 'data'
```

Scrape all links

[50]:
```
for link in soup.find_all('a',href=True):  # in html anchor/link is represented by the tag <a>

    print(link.get('href'))
```

```
https://web.archive.org/web/20230224123642/https://www.ibm.com/reports/threat-intelligence/
https://web.archive.org/web/20230224123642/https://www.ibm.com/about
https://web.archive.org/web/20230224123642/https://www.ibm.com/consulting/?lnk=flathl
https://web.archive.org/web/20230224123642/https://www.ibm.com/consulting/strategy/?lnk=flathl
https://web.archive.org/web/20230224123642/https://www.ibm.com/consulting/ibmix?lnk=flathl
https://web.archive.org/web/20230224123642/https://www.ibm.com/consulting/technology/
https://web.archive.org/web/20230224123642/https://www.ibm.com/consulting/operations/?lnk=flathl
https://web.archive.org/web/20230224123642/https://www.ibm.com/strategic-partnerships
https://web.archive.org/web/20230224123642/https://www.ibm.com/employment/?lnk=flatitem
https://web.archive.org/web/20230224123642/https://www.ibm.com/impact
https://web.archive.org/web/20230224123642/https://research.ibm.com/
https://web.archive.org/web/20230224123642/https://www.ibm.com/
```

Scrape all images Tags

[51]:
```
for link in soup.find_all('img'):
    print(link)
    print(link.get('src'))
```

```
<img alt="Person standing with arms crossed" aria-describedby="bx--image-1" class="bx--image__img" src="https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/0a23e414312bcb6f/08196d0e04260ae5_cropped.jpg.global.sr_16x9.jpg"/>
https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/0a23e414312bcb6f/08196d0e04260ae5_cropped.jpg.global.sr_16x9.jpg
<img alt="Team members at work in a conference room" aria-describedby="bx--image-2" class="bx--image__img" src="https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/06655c075aa3aa29/CaitOppermann_2019_12_06_IBMGarage_DSC3304.jpg.global.m_16x9.jpg"/>
https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/06655c075aa3aa29/CaitOppermann_2019_12_06_IBMGarage_DSC3304.jpg.global.m_16x9.jpg
<img alt="Coworkers looking at laptops" aria-describedby="bx--image-3" class="bx--image__img" src="https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/08f951353c2707b8/052022_CaitOppermann_InsideIBM_London_2945_03.jpg.global.sr_16x9.jpg"/>
https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/08f951353c2707b8/052022_CaitOppermann_InsideIBM_London_2945_03.jpg.global.sr_16x9.jpg
<img alt="Cloud developer with red sweater coding at desk" aria-describedby="bx--image-4" class="bx--image__img" src="https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/064e0139f5a3aa5e/0500002_Lowell_LI_100119.jpg.global.sr_16x9.jpg"/>
https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/064e0139f5a3aa5e/0500002_Lowell_LI_100119.jpg.global.sr_16x9.jpg
<img alt="Aerial view of automated conveyer belt and machinery at work" aria-describedby="bx--image-5" class="bx--image__img" src="https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/0795cae91a25156f/conveyorrobottopview.jpg.global.sr_16x9.jpg"/>
https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/0795cae91a25156f/conveyorrobottopview.jpg.global.sr_16x9.jpg
<img alt="Overhead view of partners collaborating on design with laptops and coffee" aria-describedby="bx--image-6" class="bx--image__img" src="https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/06dfa9ccdba4ec79/1f417900-9042-44d1-9c219a854bbb62ea.jpg.global.sr_16x9.jpg"/>
https://web.archive.org/web/20230224123642im_/https://1.dam.s81c.com/p/06dfa9ccdba4ec79/1f417900-9042-44d1-9c219a854bbb62ea.jpg.global.sr_16x9.jpg
```

Scrape data from HTML tables

[52]:
```
#The below url contains an html table with data about colors and color codes.
url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DA0321EN-SkillsNetwork/labs/datasets/HTMLColorCodes.html"
```

[53]:
```
# get the contents of the webpage in text format and store in a variable called data
data  = requests.get(url).text
```

[54]:
```
soup = BeautifulSoup(data,"html.parser")
```

[55]:
```
#find a html table in the web page
table = soup.find('table') # in html table is represented by the tag <table>
```

[56]:
```
#Get all rows from the table
for row in table.find_all('tr'): # in html table row is represented by the tag <tr>
    # Get all columns in each row.
    cols = row.find_all('td') # in html a column is represented by the tag <td>
    color_name = cols[2].string # store the value in column 3 as color_name
    color_code = cols[3].string # store the value in column 4 as color_code
    print("{}--->{}".format(color_name,color_code))
```

```
Color Name--->None
lightsalmon--->#FFA07A
salmon--->#FA8072
darksalmon--->#E9967A
lightcoral--->#F08080
coral--->#FF7F50
tomato--->#FF6347
orangered--->#FF4500
gold--->#FFD700
orange--->#FFA500
darkorange--->#FF8C00
lightyellow--->#FFFFE0
lemonchiffon--->#FFFACD
papayawhip--->#FFEFD5
moccasin--->#FFE4B5
peachpuff--->#FFDAB9
palegoldenrod--->#EEE8AA
khaki--->#F0E68C
darkkhaki--->#BDB76B
yellow--->#FFFF00
lawngreen--->#7CFC00
chartreuse--->#7FFF00
limegreen--->#32CD32
lime--->#00FF00
forestgreen--->#228B22
```